

Lookout

AI-powered privileged access misuse and insider threat detection for banking, with real-time communication security, a honeypot, and quantum-safe protection.

Author	KeerthiShree TS
College	SNS College of Technology
Live demo	https://lookout-9erd.onrender.com
Source code	github.com/keerthishree20/lookout
Date	23 September 2026

This guide is self-contained: it explains what the system does, how every part works, and why it was built this way, with the real code. All data, employees, customers and money in the system are simulated; every measured number in this document comes from that synthetic data.

Contents

1. Overview
2. Tech Stack & Why
3. Setup from Scratch
4. The Event Model
5. Behavioural Baselines
6. The Detectors
7. The Behavioural Model (IsolationForest)
8. Risk Fusion, Bands and the Graded Response
9. Insider Threat Classification
10. Session Monitoring and Containment
11. The Outbound Message Gateway
12. Quantum-Safe Cryptography
13. The Tamper-Evident Audit Log
14. The Narrator (Explainability in Prose)
15. Synthetic Data and Evaluation
16. Roles, Sign-in and Sessions
17. The Employee Portal and the Honeypot
18. The Supervised Classifier and SHAP
19. The Database
20. The API
21. The Frontend
22. Docker and CI
23. Configuration & Troubleshooting
24. Complete Feature Summary

1. Overview

Banks already guard the front door against outsiders. Insiders are harder: employees, contractors and administrators hold legitimate credentials, and their misuse looks like work. A teller whose password was phished, an analyst copying the customer table at 2 a.m., a loans officer granting themselves domain admin, a compromised account texting 50,000 customers a fake KYC link all arrive with valid credentials.

Lookout watches every action by every identity and asks one question: **is this normal for this person, given what they are allowed to do?** It answers with a score from 0 to 100, the sentences that justify the score, and a proportionate response:

Band	Score	Response
 Low	0–29	Allow and log
 Medium	30–59	Demand a second factor (risk-based authentication)
 High	60–79	Block (messages: quarantine for review)
 Critical	80–100	Block, revoke the session, page the SOC

These are the specification's bands and colours. The critical band started at 85 in the first version and moved to 80 to match the spec. The Super Admin can move all three thresholds at runtime.

Every decision goes into an audit log that is hash-chained and signed with a post-quantum signature scheme, so nobody can quietly rewrite it later, and the signatures will still hold once quantum computers can break today's schemes.

2. Tech Stack & Why

Layer	Choice	Why
API	FastAPI	Typed request models via Pydantic, WebSocket and SSE support, docs at <code>/swagger</code> and <code>/redoc</code>
Live feed	WebSocket (<code>/api/ws</code>), with SSE kept as a fallback	One connection carries decisions, stats and alerts to every console page
Models	Pydantic v2	One definition validates API input <i>and</i> serialises decisions
Anomaly detection	scikit-learn IsolationForest	Unsupervised, trains only on benign data, no labels needed
Classification	scikit-learn RandomForest + SHAP	A supervised second opinion whose every prediction can be explained
Database	PostgreSQL 16, SQLAlchemy 2, psycopg 3	A durable record of every decision; SQLite works for tests
Auth	PBKDF2 + PyJWT (HS256) + server-side session registry	Tokens that stop working the moment the engine blocks someone
Signatures	dilithium-py (ML-DSA-65, FIPS 204)	A real NIST standard, pure Python, no compiler or liboqs build
Key encapsulation	kyber-py (ML-KEM-768, FIPS 203)	Same reasons; paired with AES-256-GCM from <code>cryptography</code>
PDFs	fpdf2	Genuine and decoy exports come from the same renderer, so they can't be told apart
Narrator	Gemini over HTTPS (optional)	Prose summaries; a deterministic template is the default
Frontend	React 19 + Vite + React Router 7 + Tailwind 4 + Recharts + Axios	A multi-page console that builds to static files nginx can serve
Deployment	Docker Compose + nginx	One command starts the database, API and site
Tests	pytest (299) + Playwright browser checks	Unit, pipeline, API, roles, database, ML and a detection-quality floor

3. Setup from Scratch

The quickest way is Docker:

```
git clone https://github.com/keerthishree20/lookout
cd lookout
cp .env.example .env          # fill in JWT_SECRET, POST_QUANTUM_KEY, POSTGRES_PASSWORD
docker compose up -d --build
# open http://localhost:3000
```

Without Docker:

```
cd lookout/backend
python3.12 -m venv .venv
source .venv/bin/activate
pip install -r requirements-dev.txt
python -m pytest -q           # 299 passed
python -m lookout.evaluate    # precision / recall report
export JWT_SECRET=$(python -c "import secrets; print(secrets.token_urlsafe(48))")
uvicorn lookout.api:app --port 8077 # API + live traffic
```

```
cd ../frontend
npm ci
npm run dev                  # http://localhost:3000
```

The API uses port **8077** rather than 8000, which is usually taken by another local FastAPI project. If you change it, set `VITE_API_URL` for the frontend. Python 3.12 is required. Some machines still have an old `python3` first on the PATH, so call `python3.12` explicitly.

4. The Event Model

Everything that enters Lookout is an `Event`, and everything that leaves is a `Decision` (`backend/lookout/models.py`):

```
class Event(BaseModel):
    event_id: str
    ts: datetime
    actor: str
    actor_role: Role
    action: Action          # login, login_failed, priv_escalate, db_query, file_access,
                           # config_change, send_message, vault_read, logout

    resource: str = ""
    source_ip: str = "0.0.0.0"
    device_id: str = "unknown"
    geo: Geo
    success: bool = True
    message: MessagePayload | None = None
    meta: dict[str, Any]      # record_count, target_role, session_id, bytes_written ...
    label: ThreatClass | None = None # ground truth, generated data only
```

Roles map to privilege levels 1–6, from teller to domain admin. Two frozensets define who may speak to customers at all, and who may do it in bulk:

```
CUSTOMER_COMMS_ROLES = frozenset({Role.OFFICER, Role.MANAGER, Role.DOMAIN_ADMIN})
BULK_COMMS_ROLES     = frozenset({Role.MANAGER, Role.DOMAIN_ADMIN})
```

Why a `label` field that detectors never read? It makes evaluation possible (section 15). The API strips it from every response so nobody inspecting the console's traffic has to take that on trust.

5. Behavioural Baselines

A 2 a.m. login is routine for the on-call sysadmin and alarming for a teller. So every threshold is measured against the actor's own history (`baselines.py`):

```
class Baseline:
    hour_counts: list[int]          # 24-bin activity histogram
    countries, devices, ip_prefixes # categorical "seen before" sets
    records_per_query: RunningStat  # Welford online mean / std
    recipients_per_message: RunningStat
    last_login_ts, last_login_geo    # for impossible travel
```

`RunningStat` uses Welford's algorithm, so the profile updates one event at a time with no stored history. Its z-score floors sigma at `max(1, 10% of mean)` . Without the floor, a teller whose queries are always exactly 40 rows would get an infinite z-score the first time they read 41.

Why only learn from allowed events? In `pipeline.py` :

```
if action == ActionTaken.ALLOW:
    baseline.observe(event)
```

If a blocked 412,000-row read fed into the baseline, the next exfiltration would look less unusual. A test checks the baseline mean is unchanged after the exfiltration scenario.

6. The Detectors

Twenty-seven detectors, each a function `(event, baseline, ctx) -> list[Signal]`. A `Signal` carries its points, a complete sentence, and which threat classes it is evidence for:

```
Signal(  
  name "impossible_travel",  
  points 50.0,  
  explanation "Login from Kyiv, UA is 6,102 km from the previous login in Chennai "  
              "28 minutes earlier -- 13,076 km/h, which no traveller can achieve. ...",  
  indicates [ThreatClass.COMPRROMISED],  
)
```

Group	Detector	Fires when
Identity	<code>impossible_travel</code>	great-circle distance ÷ elapsed time > 900 km/h
	<code>abnormal_login_time</code>	the hour holds < 1% of this person's history
	<code>new_device_or_network</code>	first sighting of a device, country or /24 network
	<code>failed_login_burst</code>	≥ 5 failures in 15 min (more points from an unfamiliar origin)
	<code>credential_misuse</code>	a success after ≥ 5 failures
	<code>containment_breach</code>	revoked session reused, locked origin retrying, or persisting after a block
Privilege	<code>privilege_escalation</code>	requested role above held role; scored by levels jumped + burst
	<code>out_of_scope_admin_action</code>	config change / vault read / escalation by a role without the mandate
	<code>dormant_privileged_account</code>	level ≥ 4 account active after 45+ idle days
Data	<code>mass_record_access</code>	rows read far above personal norm, or ≥ 10,000 absolute
	<code>off_hours_data_access</code>	data access outside 08:00–20:00
	<code>bulk_file_write</code>	≥ 250 MB written in one operation (data staging)
	<code>vault_hoarding</code>	≥ 8 vault reads in 30 minutes
Messaging	<code>suspicious_url</code>	a link impersonates the bank or hides its destination
	<code>bulk_message_blast</code>	recipients far above sender's norm, or from an unauthorised role
	<code>unauthorized_customer_comms</code>	customer messaging from a role without the mandate
Transfers	<code>suspicious_transfer</code>	over the role's limit or far above personal norm, a new outside payee, out of hours; a role with no transfer mandate is blocked outright
Sign-in	<code>compound_login_anomaly</code>	three or four of {rare hour, new device, new country, new network} at once
Sessions	<code>concurrent_sessions</code>	a second live session from another device or network

Group	Detector	Fires when
	login_frequency	five or more sign-ins in an hour
	session_context_change	one session's requests suddenly from a different device or /24 network (a stolen token)
	query_rate_burst	60+ database queries in five minutes (the spec's "10 an hour, then 500 in 5 minutes")
	failed_authorization	refused requests; three in 30 minutes means someone is probing
Content	phishing_language	the NLP content model says the text reads like a scam (section 11)
	sensitive_data_leak	card (Luhn-checked), Aadhaar, PAN, IFSC, account numbers or credentials in the text; worse to a personal mailbox
	risky_attachment	executables, double extensions (statement.pdf.exe), macro documents, HTML files, large data exports
	repeated_message	the same <i>suspicious</i> text sent three or more times in an hour

Why a compound sign-in detector

The spec says a sign-in at 02:30 from a new device, a new country and an unknown IP "should generate a high-risk login". Scored as four separate weak signals, it reached 34: medium. The spec's own example scenario also says a new device at 02:30 should get *additional verification*, not a block. Both are right: each novelty alone is ordinary (people get new laptops and travel), but all of them at once is how a stolen password looks. So the compound detector adds nothing for one or two novelties, 12 points for three and 30 for four. Now four novelties score 66.5 (HIGH, blocked into the honeypot) and the spec's step 2 scores 46.9 (MEDIUM, a second factor). Benign traffic never has three at once, so the false-positive count stayed at zero.

Why logarithmic volume scoring

An early version scored volume linearly in z . A test caught the flaw: a teller whose queries are always about 40 rows produces $z \approx 3,000$ on *any* big read, so 12,000 rows and 400,000 rows both hit the cap and scored the same. The fix uses two log-scaled terms:

```
z_term = min(12.0, 4.0 * math.log2(max(z, 4.0) - 4.0)) # unusual for *them*
magnitude = math.log10(max(count, 1) - VOLUME_ABSOLUTE_FLOOR)
floor_term = 10.0 - 12.0 * max(magnitude, 0.0) if over_floor else 0.0 # a lot, full stop
points = min(46.0, 14.0 + z_term + floor_term)
```

bulk_message_blast had the same bug and got the same fix, so 900 and 50,000 recipients now score differently. Both have regression tests.

7. The Behavioural Model (IsolationForest)

Rules catch the misuse someone thought to describe. `anomaly.py` catches the rest. Each event becomes a 13-number vector, every feature *relative to the actor's baseline*: hour rarity, unseen country/device/network, privilege level, log rows, log recipients, worst URL score, recent failures, log km from last login, events in 30 min, action sensitivity, failed flag.

```
raw    float(self._forest.decision_function(vector.reshape(1, 1))[0])
score  1.0    (1.0 - math.exp(raw - 12.0))          # 0..1, 1 = most anomalous
points MAX_MODEL_POINTS    score                  # capped at 22 of 100
```

Attributions are the three features furthest from the training mean in standard deviations, so the model's contribution also carries a sentence: *"driven by log recipients (+8.80 vs typical)"*.

Why cap the model at 22 points? An unsupervised model that cannot fully explain itself should never be the sole reason an employee is locked out. It can tip a borderline case; it cannot create one.

Why train only on benign data? A model trained on attacks learns to call attacks normal. `train_from_history` skips any event with a non-benign label.

8. Risk Fusion, Bands and the Graded Response

```
total = min(100, (rule_points + model_points) × (1 + 0.07 × (privilege_level - 1)))
```

The privilege multiplier runs from 1.00 for a teller to 1.35 for a domain admin. The same behaviour is not equally dangerous: a domain admin can do more with 50,000 rows than a teller can.

`decide()` maps the band to a response, then applies **policy floors**:

```
if to_customers and hostile_link and _below(action, ActionTaken.QUARANTINE):  
    return ActionTaken.QUARANTINE, HOSTILE_LINK_POLICY
```

Why a floor for hostile links? A browser test found the gap: one lookalike-domain link to one customer scored 30, which meant step-up auth. If the sender passed MFA, the link went out. A malicious insider passes their own MFA without effort, so step-up can't be the control here. Any customer-facing message with a suspicious link is now held for human review, and the decision carries the policy text so the analyst can see why a score of 30 was quarantined.

Risk-based authentication. The strength of the step-up scales with privilege and score:

```
if event.privilege == 5 or risk.total == 50: return "hardware_security_key"  
if event.privilege == 3 or risk.total == 40: return "totp_and_manager_approval"  
return "push_notification"
```

9. Insider Threat Classification

Each signal lists the classes it supports, **primary first**. The primary class gets the signal's full points and each secondary class gets half:

```
for i, cls in enumerate(signal.indicates):
    weights[cls] += signal.points * (1.0 if i == 0 else 0.5)
```

Without the weighting, an escalation attempt ([PRIVILEGE_ABUSE, MALICIOUS]) tied and the severity tie-break labelled it malicious.

Two more rules:

- **Containment signals don't vote.** "They kept going after being blocked" raises risk but says nothing about *what kind* of insider this is. Those events inherit the class their session already has.
- **"Compromised" is sticky.** The classes describe the employee, not the act. Once a session is under someone else's control, everything done in it belongs to the intruder. Labelling the employee "malicious" for the intruder's query would be wrong.

The negligent-insider scenario tests the distinction. A teller emails 900 customers a *genuine* link.

`bulk_message_blast` classes volume with a clean link as `NEGLIGENT` and volume with a hostile link as `MALICIOUS`. The result is quarantine, never a SOC page.

10. Session Monitoring and Containment

`context.py` holds the consequences of earlier decisions:

```
self.revoked_sessions: set[str]
self.locked_origins: set[tuple[str, str]]      # (actor, source_ip)
self.session_strikes: dict[str, int]          # blocks/holds already issued
self.session_class: dict[str, ThreatClass]
```

`pipeline._respond()` updates it after each decision:

- BLOCK_AND_ALERT, or BLOCK on a *login*, revokes the session
- BLOCK_AND_ALERT on a login attempt locks the source IP out of that account
- any block or quarantine is a strike against the session

The bug this fixed. In the credential-stuffing scenario the Lagos login was blocked, but only BLOCK_AND_ALERT revoked sessions, so the nine vault reads that followed on the same session were *allowed*. A refused login now leaves no usable session, and a regression test covers it.

11. The Outbound Message Gateway

`POST /api/messages/scan` is where every staff SMS, email or push is checked before it leaves. URLs are extracted from the **body** rather than taken from a separate field, because a sender hiding a link would simply leave the field empty.

The first version only matched links starting with `http://`, `https://` or `www.`. Writing the API docs exposed the gap: the example phishing SMS, `Update at meridian-bank.secure-verify.top/re-kyc`, came back with no links at all, and real phishing texts usually leave the scheme off. The fix also catches bare domains, but only on a known TLD, so ordinary text doesn't turn into links:

```
_BARE = r"(?![\w@.-])(?:[a-z0-9](?:[a-z0-9-]{0,61}[a-z0-9])?\.\.)+([a-z]{2,24})(?![\w-])\n(?:/[^\s<>\"'"]*)?"

def extract_urls(text):
    for m in _LINK.finditer(text):
        tld = m.group(1)
        if tld is not None and tld.lower() not in LINK_TLDS:
            continue # "today.Update" and "report.pdf" are not links
    ...
```

The lookbehind stops `ops@meridianbank.in` from matching as a link, and `"Rs.500"` fails because `500` isn't a TLD.

`urlcheck.py` decides offline whether a link is hostile. With no threat-intel feed, it asks the question that matters for internal phishing, **does this link claim to be us when it isn't?**:

```
HOMOGLYPHS = {"0": "o", "1": "l", "3": "e", "4": "a", "5": "s", "7": "t", ...}

def _fold(text): # visual twins become string twins
    folded = "".join(HOMOGLYPHS.get(c, c) for c in text.lower())
    return folded.replace("rn", "m").replace("vv", "w").replace("-", "")
```

After folding, `levenshtein(host, corporate) ≤ 2` flags lookalikes such as `meridianbamk.com`, `meridlanbank.com`, `rneridianbank.com` and `meridian-bank.com`. Other findings: brand embedded under a foreign domain, shorteners, bare IPs, punycode, abuse-heavy TLDs, credential bait (`kyc`, `otp`, `verify` ...), plain HTTP, deep subdomain chains.

URLs get lexical features too: length, a `user@host` disguise (`https://meridianbank.com@login-verify.top` really goes to `login-verify.top`), percent-encoding, redirect parameters (`?url=https%3A...`), hyphen- or digit-stuffed hosts. Each link gets a 0-100 **URL risk**. Domain age and live redirect chains need a network lookup, which Lookout deliberately doesn't do.

Reading the words: the content model

A scam doesn't need a link: "This is the fraud team, reply with the OTP you received". The spec asks for "an ML/NLP model where appropriate" rather than keywords, so `nlp.py` trains a TF-IDF (word 1-2 grams) + logistic regression model on 3,000 template messages, about 30% scams. The explanation names the phrases that pushed the probability up:

```

row    self.pipeline.named_steps["tfidf"].transform([text])
contributions    row.multiply(self._coef).tocsr()          # TF-IDF weight x coefficient
idx    contributions.indices[np.argsort(contributions.data)[:, : 1]]
phrases    [self._vocab[i] for i in idx if self._coef[i] > self._strong][:top]

```

The first version flagged "Hi team, lunch at 1?" at 0.53, because short neutral text sat near the decision boundary. Adding ordinary staff chat to the corpus moved it to 0.07. The detector fires at 0.7. The corpus is synthetic and the held-out F1 of 1.0 only proves the pipeline runs.

Other content checks: `sensitive_data_leak` counts PII by kind and never repeats the values (a test caught the first 12 digits of a card number also counting as an Aadhaar number); `risky_attachment` reads attachment metadata; `repeated_message` catches a campaign split into batches, but only when the text is suspicious, since "your statement is ready" repeats all day.

The message risk score

The spec's formula is URL + sender behaviour + content + destination + privilege + volume. Lookout doesn't compute a second, separate score: `message_risk.py` splits the one score that decided the message into those six parts, moving the privilege multiplier's extra into "privilege" and the personal-mailbox points into "destination", so the parts always add up to the total.

The graded outcomes: deliver, hold for step-up, quarantine, or block and page. A held message can be **released**, **blocked** (confirmed bad, and a strike against the sender's session), **deleted**, or put under **investigation** (still held, and filed in the sender's incident). All four are audited. Who may message customers at all, and who may run bulk campaigns, is a configurable communication policy; a privileged administrator messaging customers always needs a second factor.

12. Quantum-Safe Cryptography

Two jobs, two NIST standards (`crypto.py`):

Job	Algorithm	Threat it answers
Sign the audit log	ML-DSA-65 (FIPS 204)	Ed25519/ECDSA signatures become forgeable once a large quantum computer exists, which retroactively destroys a log's evidential value
Seal credentials	ML-KEM-768 (FIPS 203) + AES-256-GCM	Harvest now, decrypt later: vault blobs stolen today must stay unreadable

```
def seal(self, plaintext, aad b''):
    shared, kem_ct = ML_KEM_768.encaps(self._ek)
    nonce = os.urandom(12)
    ct = AESGCM(_derive_key(shared)).encrypt(nonce, plaintext, aad)
    return SealedBlob(b64(kem_ct), b64(nonce), b64(ct), self.algorithm)
```

The spec asks for protected credentials, configuration, key material and audit artefacts, kept clearly apart from the classical crypto that signs people in. `pq_vault.ProtectedStore` seals, at start-up, the JWT secret, the database URL, synthetic PAM-vault credentials, a policy snapshot and the audit-signing seed itself (key wrapping), and stores a SHA-256 digest of each. "Verify every artefact" reopens them all. The console's "Corrupt" button flips one ciphertext byte; AES-GCM refuses to open it, and Lookout raises a **Quantum-Safe Key/Artefact Security Event**, which pops up on every SOC screen over the WebSocket. A table on the same page lists what each layer uses and whether it is classical or post-quantum. PBKDF2, HMAC and SHA-256 are listed as classical.

The vault path is the AES-GCM associated data, so a sealed blob moved to a different path won't decrypt (tested). Both schemes sit behind small interfaces with classical fallbacks. On a host without the PQC libraries the dashboard shows "classical" rather than claiming quantum safety.

Why pure-Python implementations? `liboqs-python` compiles liboqs from source at import time. `dilithium-py` and `kyber-py` install with pip in seconds and implement the same standards. The cost is speed (about 90 ms per ML-DSA signature), which is why the audit log signs checkpoints rather than every entry.

13. The Tamper-Evident Audit Log

```
entry_hash = sha256(canonical({"seq", "ts", "kind", "payload", "prev": prev_hash}))
```

Each entry links to the one before. Every 25 entries, and immediately after anything critical, the chain head is **signed with ML-DSA**. Verification recomputes the chain and checks every signature. Three attacks are tested:

Attack	Caught by
Edit a payload, leave the hash	content no longer matches its hash, reported at that entry
Edit a payload and recompute its hash	the next entry's <code>prev_hash</code> no longer links
Rewrite the whole chain consistently	the signed checkpoint no longer verifies without the private key

The console's **Forge entry** button runs the first attack live. **Reset demo** restores state.

14. The Narrator (Explainability in Prose)

The score, class and action are all fixed before `narrator.py` runs. The narrator only writes them up. With no key it builds the summary from the top two signals:

p.nair (officer) scored 30/100 for sending an SMS to 1 recipient -- medium risk, consistent with deliberate misuse. Outbound SMS to 1 recipient contains <https://meridianbank.com/login> -- host resembles the corporate domain meridianbank.com ... Held by policy: Customer-facing messages carrying a suspicious link are never delivered without human review, whatever the sender's risk score.

With `GEMINI_API_KEY` set, Gemini receives the same evidence and is told to invent nothing and neither soften nor escalate. If a call fails once, the narrator marks itself degraded and uses the template from then on, so a network outage never stalls decisions.

15. Synthetic Data and Evaluation

Real privileged-access logs from a bank can't be obtained, and an approximation scraped from elsewhere would be worse than none. `generator.py` simulates **Meridian Bank**: 12 staff across Chennai, Bengaluru, Mumbai and Singapore, each with their own hours, devices, query volumes and messaging habits. The simulation includes the awkward benign cases: managers running 400–2,500-recipient campaigns, weekend on-call admins, one or two mistyped passwords a day.

`evaluate.py` trains on 30 days and scores a held-out week plus all the scenarios:

```
precision      : 1.000
recall         : 0.976
false positive rate : 0.0000 (0 of 873 benign)
threat classification : 1.000 (40 of 40 detected incidents named correctly)
policy step-ups  : 8 (low-score events given MFA because policy requires it)
```

Policy step-ups are counted separately on purpose. The spec says privileged administrators need a second factor for customer messages, so n.pillai's ordinary messages now get one. Counting those as false positives would punish a mandatory control. Counting them as detections, when an attack only reaches a policy step-up, would inflate recall. So the evaluator reports them on their own line and scores neither way.

The one miss is on purpose: the 02:14 login that opens the exfiltration scenario is allowed, because an unusual hour alone isn't enough to lock someone out. The query four minutes later is blocked.

What this does and doesn't prove. Lookout generated both the attacks and the normal traffic, so these numbers show internal consistency and that ordinary work doesn't trigger alerts. They are **not** a claim about a real bank.

`tests/test_evaluation.py` sets floors below today's numbers so that a later change can't quietly trade recall for fewer alerts.

The generator also had a bug of its own: simulated tellers sent customer messages, which the policy forbids, so "benign" data broke the rules it was scored against. That was the only false positive. Fixing the simulation, not the detector, was the right call.

16. Roles, Sign-in and Sessions

There are five roles:

Kind	Accounts	Can
Employee	12 staff, e.g. <code>r.krishnan</code> / <code>Teller@Krishnan1</code>	the portal: customers, transfers, access requests, own profile and activity, open protected systems their role allows
Manager	<code>l.mathew</code> , <code>v.rao</code> (employees with the manager role)	the portal, plus My team : their team's risk, and approving access requests
Privileged administrator	<code>t.banerjee</code> (DBA), <code>h.queshi</code> (sysadmin), <code>n.pillai</code> (domain admin)	the portal, plus the Admin console : disable or enable accounts and change roles <i>below their own level</i> , edit the communication policy. Every operation is a scored admin event
SOC analyst	<code>soc.analyst</code> / <code>SocWatch@2026</code>	the whole console
Super admin	<code>super.admin</code> / <code>SuperAdmin@2026</code>	the console, plus live policy and final access approval

Passwords are PBKDF2-HMAC-SHA256 with a per-account salt. A successful sign-in returns an HS256 JWT, but the token isn't the whole story: it carries a session ID that is checked against a server-side registry on every request. That is what makes revocation instant. When the engine blocks a login or the SOC clicks **Revoke**, the session is gone, even though the JWT is still unexpired.

Access control is a single table, checked once in middleware instead of on every handler:

```
ROUTE_ACCESS = (
    ("/api/health", PUBLIC),
    ("/api/auth/", PUBLIC),
    ("/api/portal/", frozenset({"employee"})),
    ("/api/team/", frozenset({"employee"})), # and the handler checks for a manager
    ("/api/admin/", frozenset({"superadmin"})),
)
CONSOLE = frozenset({"soc", "superadmin"}) # everything else
```

Because the default is "console only", a new route can't accidentally be exposed to employees.

The sign-in is also an **event**. The engine scores it like anything else (new device, impossible travel, a burst of failures), and a blocked login returns 403 with no token even when the password was right. On top of that, `LoginLimiter` refuses more than 10 failures in 5 minutes for an account, or more than 60 attempts a minute from one IP, with a 429.

What the sign-in returns depends on its risk. LOW: a session, plus `{risk_score, risk_level, reason[]}`. MEDIUM: no session yet, a simulated one-time code to confirm at `/api/auth/mfa/verify`, and each wrong code scored as a failed sign-in. HIGH: a session, but into the honeypot. The sign-in page has a **Sign-in context** selector (new

laptop at 02:30; 02:30 from a new country on an unknown device; New York), because a demo can't really change your device or city. The server applies it to the sign-in event: device, IP, city and hour.

Privilege escalation, the spec's way. If a teller calls an admin-console route, they get a 403, and the attempt is ingested as a `PRIV_ESCALATE` event aimed at domain admin. The escalation detector scores it, and the teller likely lands in the honeypot. A refusal that isn't also a signal would let someone probe for free.

Bug found by the browser test: a blocked login used to leave a usable session behind, so the "blocked" attacker could still read the vault. The fix revokes the session whenever the response is a block, and a regression test keeps it that way.

17. The Employee Portal and the Honeypot

The employee portal has four tabs: **Customers** (the book, with PII masked), **Fund transfer**, **Access requests**, and, for managers, **My team**.

The brief asked for something unusual: when an employee does something risky, don't block them visibly. Show them "a duplicate page of exactly how the transfer page will look", with fake values, record everything, and tell the monitoring team.

The key design decision was that there is **no duplicate page**. A second page, however faithful, could differ in a detail an insider would notice. Instead, the *same* pages switch their data source. An engine listener watches every decision:

```
def _honeypot_trigger(self, decision):
    high = decision.risk.band in (Band.HIGH, Band.CRITICAL) or decision.action_taken in (
        ActionTaken.BLOCK, ActionTaken.BLOCK_AND_ALERT)
    if not high or actor not in BY_ACTOR:
        return
    if self.ledger.watch(actor, reason):
        self.incidents.note_for_user(actor, "honeypot", f"Moved into the honeypot: {reason}. ...")
        self.engine.audit.append("honeypot.activated", {...}, critical=True)
```

Once someone is on the watchlist:

- **Transfers** go to a per-employee **shadow ledger** (`banking.py`). The OTP step, the "Transfer successful" message, the reference number, the UTR and the debited balance all behave exactly like the real thing, and the fake balance stays debited on refresh. No real money moves; each fake transfer is audited with `real_funds_moved: false`.
- **Exports** return a decoy PDF from the same renderer as a real one. Everything the employee saw on screen is kept (names, customer IDs, the last four digits of the account number), and everything the screen masked is fabricated. A decoy that disagreed with the screen would give itself away.
- **Canaries**. Fake account numbers, the PDF's footer reference (`MB-DOC-...`), and fake transaction references and UTRs are all recorded. If one turns up later, the SOC pastes it into **Trace** and gets the name of the employee who received it.

Exporting 100 or more customers triggers a decoy on its own; the super admin can change the threshold.

Two bugs worth knowing about. The watchlist was first keyed by session, so a caught employee who signed out and back in got real data again, and could compare it with the decoy. It is now keyed by person, and only the SOC's **Clear** ends it. Second, the decoy PDFs' footer was drawn after the table, and on a full page that pushed it onto a new page that genuine exports didn't have. Moving it into FPDF's `footer()` hook fixed that.

Managers see their team's risk but are never told that someone is in the honeypot. The browser test checks that the words don't appear anywhere on the manager's page.

My profile shows the employee their own details and recent activity, and lets them open protected systems. It never shows risk scores, which would teach an insider where the thresholds are. Opening a system runs the same policy engine as the SOC's access check: out of role is **denied** and logged as a failed authorisation; medium current risk, or a privileged account opening a critical system, needs a **second factor**; high risk gets **read-only** access.

18. The Supervised Classifier and SHAP

The spec asked for a trained model with proper evaluation. The IsolationForest (section 7) is unsupervised; this one learns from labels.

Dataset. `ml/generate_dataset.py` writes 10,600 synthetic sessions to `backend/datasets/insider_sessions.csv`: 9,400 normal (88.7%), 420 negligent, 330 malicious, 260 compromised and 190 privilege abuse. There are 18 features per session: login hour, location and device change, failed logins, data volume, sensitive-resource access, message count, suspicious URLs, privilege level, role-change attempts, transfer amount, new outside payees and so on.

The first version of the dataset had two shortcuts, and the model found both. "Any escalation request" was a perfect privilege-abuse label, because no normal session contained one. And "short session" identified every compromise; SHAP showed the model leaning on exactly that one feature. Real life overlaps, so the generator was changed:

- admins make approved just-in-time elevations for change windows;
- half of the compromised sessions are hijacks in the middle of the real person's working day, so the session keeps its ordinary length and content;
- negligent sessions are mostly ordinary work.

Model. `RandomForestClassifier(n_estimators=200, min_samples_leaf=3, class_weight="balanced_subsample")`. Without the class weighting, predicting "normal" every time would score 88.7% accuracy.

Evaluation, done two ways:

	stratified 25% split	unseen employees
accuracy	0.973	0.923
macro F1	0.903	0.867
threat recall	0.837	0.876
negligent recall / precision	0.638 / 0.827	0.677 / 0.297

The second column is the honest one. Tested on employees it has never seen, the model confuses careless-but-innocent with ordinary work: seven in ten of its "negligent" calls are wrong. That is why the model gives a **second opinion** next to the rules and never decides on its own.

SHAP. For every flagged session, `TreeExplainer` returns the features that pushed the prediction towards its class. The console's decision panel shows them next to the rule signals, so an analyst can see both why the rules fired and what the model thinks.

```
cd ml && python generate_dataset.py && python train.py && python explain.py
```

19. The Database

With `DATABASE_URL` set, every decision is written through to PostgreSQL across 15 tables: roles, permissions, users, login events, sessions, activity logs, risk scores, threat events, messages, URL analysis, quarantined messages, alerts, incidents and audit logs. Persistence is an engine **listener**, so the detection code doesn't know the database exists.

Each API start is a *run* with its own `run_id`, because the in-memory audit chain starts again at sequence 0. `GET /api/db/status` recomputes the SHA-256 chain **from the database rows**, which catches someone editing the table directly, not just the in-memory log.

Bug found by the tests: SQLite hands back timestamps without a timezone and PostgreSQL with one, so comparisons between the two failed. Every timestamp read back now goes through a `_utc()` helper. CI runs the database tests against a real PostgreSQL service as well as SQLite.

What it doesn't do: the engine rebuilds its baselines from the synthetic replay on each start and doesn't reload them from the database. The database is the durable record, not the engine's working memory.

20. The API

About 70 routes; the full list is in `docs/api.md` and at `/swagger` . The main groups:

Group	Examples
Auth	<code>POST /api/auth/login</code> , <code>/logout</code> , <code>GET /api/auth/me</code>
Employee portal	<code>/api/portal/customers</code> , <code>/export</code> , <code>/transfers</code> , <code>/transfers/verify</code> , <code>/access-requests</code>
Manager	<code>/api/team/overview</code> , <code>/api/team/access-requests/{id}/decide</code>
Super admin	<code>GET/PUT /api/admin/policies</code> , <code>/api/admin/access-requests</code>
Live	<code>WS /api/ws</code> , <code>GET /api/stream</code> (SSE)
Detection	<code>/api/decisions</code> , <code>/api/events</code> , <code>/api/scenarios/{key}/run</code> , <code>/api/risk/{id}/explanation</code>
Messaging	<code>/api/messages/scan</code> , <code>/api/urls/inspect</code> , <code>/api/quarantine</code>
SOC work	<code>/api/alerts</code> , <code>/api/incidents</code> (assign, notes, status, actions), <code>/api/sessions/{id}/revoke</code> , <code>/api/accounts/{u}/disable</code>
Honeypot	<code>/api/honeypots</code> , <code>/api/honeypots/trace</code> , <code>/api/honeypots/watchlist/{actor}/clear</code>
Audit, crypto, ML, DB	<code>/api/audit/verify</code> , <code>/api/crypto</code> , <code>/api/ml/metrics</code> , <code>/api/db/status</code>

21. The Frontend

React 19 with Vite, React Router and Tailwind 4. The routes:

- `/login` lists the demo accounts.
- `/employee` is the portal.
- The console has 17 pages behind a sidebar (`layouts/SocLayout.tsx`): Dashboard, Simulation, Live activity, Alerts, Incidents, Honeypot, Users (with a per-user page), Sessions, Access control, Privileged access, Messages, Message scanner, Quarantine, Audit & quantum-safe, ML insights, Security policies and Settings.

`hooks/useLive.tsx` opens one WebSocket per tab and shares decisions, stats and alerts with every page. The session token lives in `sessionStorage`, so an employee and the SOC can be signed in side by side in two tabs of the same browser, which is how the honeypot demo is run.

A Playwright script checks it end to end: all 17 console pages load, the WebSocket connects, SHAP rows appear, an incident can be assigned, the decoy PDF and fake transfer look genuine to the employee and show up for the SOC, the manager never sees the honeypot, the super admin can save policy, the database audit chain verifies, and nothing overflows at phone width.

Two bugs it caught. The super admin's **Save policy** did nothing: the browser's CORS preflight rejected `PUT` because only GET and POST were allowed. Second, the sign-in page overflowed a phone screen by 554 px, because a CSS grid column sized itself to a long table. `grid-cols-[minmax(0,1fr)]` fixed it.

The long tables (alerts, incidents, messages, quarantine, audit log) have search, filters and pagination through a shared `usePaged` hook, and details open in a modal dialog. **Bug:** the quarantine dialog first rendered clipped inside its card, because a card's `backdrop-blur` creates a containing block for `position: fixed` children. Rendering dialogs through a portal on `document.body` fixed it. The screenshot check caught this, where a "dialog opened" assertion had passed.

22. Docker and CI

```
docker compose up -d --build
db  postgres:16-alpine, its own volume, port not published
api python:3.12-slim; trains the classifier during the build; healthcheck on /api/health
web  node build → nginx; serves the app, proxies /api and the WebSocket to api
```

The frontend is built with `VITE_API_URL=""`, so it calls `/api/...` on its own origin. nginx forwards those calls, including the WebSocket upgrade, which means CORS never comes into it. All three containers use `restart: unless-stopped`, so the app comes back after a reboot. Compose refuses to start without a `JWT_SECRET`.

GitHub Actions runs three jobs on every push: the backend tests (against a PostgreSQL service container), the frontend build, and a Docker image build.

23. Configuration & Troubleshooting

Variable	Default	Effect
<code>DATABASE_URL</code>	empty (in-memory)	PostgreSQL connection
<code>JWT_SECRET</code>	random per process	signs session tokens; 32+ characters
<code>POST_QUANTUM_KEY</code>	demo seed	stable ML-DSA key across restarts
<code>FRONTEND_URL</code> , <code>CORS_ORIGINS</code>	empty	allowed browser origins
<code>MODEL_PATH</code>	<code>backend/trained_models/insider_rf.joblib</code>	classifier location
<code>GEMINI_API_KEY</code>	empty	enables LLM narratives
<code>LOOKOUT_HONEYPOT_THRESHOLD</code>	<code>100</code>	customers per export before a decoy
<code>LOOKOUT_LIVE_TRAFFIC</code>	<code>1</code>	background benign activity
<code>LOOKOUT_ALLOW_TAMPER</code>	<code>1</code>	demo forgery endpoint
<code>LOOKOUT_SHOW_DEMO_ACCOUNTS</code>	<code>1</code>	list demo credentials on the sign-in page
<code>VITE_API_URL</code>	<code>http://localhost:8077</code>	frontend → API, fixed at build time

Symptom	Cause / fix
"Cannot reach the Lookout API"	API not running, or on another port: set <code>VITE_API_URL</code> and rebuild
Everyone is signed out after a restart	<code>JWT_SECRET</code> isn't set, so a random one is generated each start
<code>SyntaxError: future feature annotations</code>	an old <code>python3</code> (3.6) ran it; use <code>python3.12</code>
Dashboard shows "classical" crypto	<code>dilithium-py</code> / <code>kyber-py</code> aren't installed in this venv
Credential stuffing looks different the second time	intended: the attacker's IP is still locked. Use Reset
An employee keeps getting fake data	they're in the honeypot; the SOC clears them on the Honeypot page

24. Complete Feature Summary

Feature	File
Per-identity online baselines	<code>baselines.py</code>
27 explainable detectors: sign-in, session, privilege, data, message links and content, transfers	<code>rules/</code>
NLP message-content model (synthetic corpus)	<code>nlp.py</code>
Six-part message risk breakdown; 0-100 URL risk	<code>message_risk.py</code> , <code>urlcheck.py</code>
IsolationForest with attributions, capped contribution	<code>anomaly.py</code>
Privilege-weighted fusion, live policy bands, graded response	<code>scoring.py</code> , <code>policy.py</code>
Policy floors: hostile customer links reviewed, no-mandate transfers blocked	<code>scoring.decide</code>
Insider classification, sticky compromise	<code>scoring.classify</code>
Session revocation, origin lock-out, persistence detection	<code>context.py</code>
Offline URL reputation and scheme-less link extraction	<code>urlcheck.py</code>
Message gateway; quarantine release / block / delete / investigate	<code>api.py</code> , <code>routes_spec.py</code> , <code>pipeline.py</code>
Five roles, JWT + revocable sessions, rate limiting, simulated MFA on sign-in	<code>auth.py</code> , <code>api.py</code>
Employee portal: customers, transfers, access requests, own profile and activity, PAM requests, team view, admin console	<code>portal.py</code> , <code>banking.py</code> , <code>routes_spec.py</code>
Post-quantum protected store with key wrapping; quantum-safe integrity alerts	<code>pq_vault.py</code>
Super Admin roles and system configuration	<code>routes_spec.py</code> , <code>runtime.py</code>
Honeypot: decoy PDFs, shadow-ledger transfers, canary tracing	<code>portal.py</code> , <code>banking.py</code>
Alerts and incident management	<code>incidents.py</code>
RandomForest classifier on 10,600 sessions, with SHAP	<code>lookout/ml/</code> , <code>ml/</code>
PostgreSQL write-through, 15 tables, chain re-verified from rows	<code>lookout/db/</code>
ML-DSA-65 audit checkpoints, ML-KEM-768 credential sealing	<code>crypto.py</code> , <code>audit.py</code>
Template / Gemini narrator	<code>narrator.py</code>
9 labelled scenarios, including the spec's attack story	<code>scenarios.py</code>

Feature	File
React + Vite console, 17 pages, WebSocket events for alerts, incidents, risk, messages and sessions	<code>frontend/</code>
Section-by-section spec coverage	<code>docs/spec-coverage.md</code>
Docker Compose + nginx; CI with PostgreSQL and image builds	<code>docker-compose.yml</code> , <code>.github/workflows/ci.yml</code>
299 tests	<code>backend/tests/</code>

Tech Stack at a Glance

```

Backend  Python 3.12 · FastAPI · Pydantic v2 · SQLAlchemy 2 · PostgreSQL 16 · PyJWT
ML       scikit-learn (IsolationForest, RandomForest, TF-IDF + logistic regression) · SHAP · pandas
        · numpy · joblib
Crypto   ML-DSA-65 (dilithium-py) · ML-KEM-768 (kyber-py) · AES-256-GCM · HKDF · Ed25519/X25519
fallback
Frontend React 19 · Vite · React Router 7 · Tailwind CSS 4 · Recharts · Axios · lucide-react
Deploy   Docker Compose · nginx
Testing  pytest (299) · Playwright (browser checks, run from outside the repo) · GitHub Actions

```